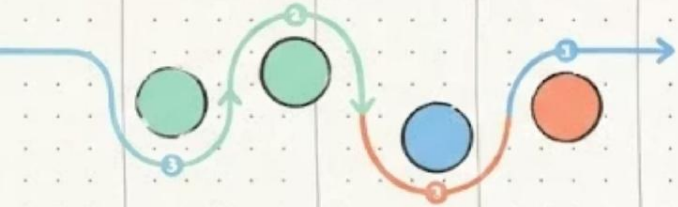


Gestão do Ciclo de Vida da Aplicação

Testes

Professor Romário da Silva Borges



```
public class Math {

  public int fatorial(int n) {
    int fat = 1;
    for (int i = 1; i <= n; i++)
      fat = fat * i;
    return fat;
  }
}
```

Qual código falta aqui?

```java
public class Math {

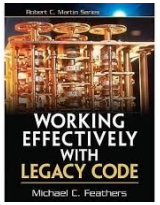
  public int fatorial(int n) {
    int fat = 1;
    for (int i = 1; i <= n; i++)
      fat = fat * i;
    return fat;
  }
}
```

```java
public class MathTest {

  @Test
  public void testFatorial() {
    Math m = new Math();
    assertEquals(1, m.fatorial(0));
    assertEquals(1, m.fatorial(1));
    assertEquals(2, m.fatorial(2));
    assertEquals(6, m.fatorial(3));
  }
}
```

"Testes de unidade são amplamente usados no Google. Todo código de produção deve ter testes de unidade."

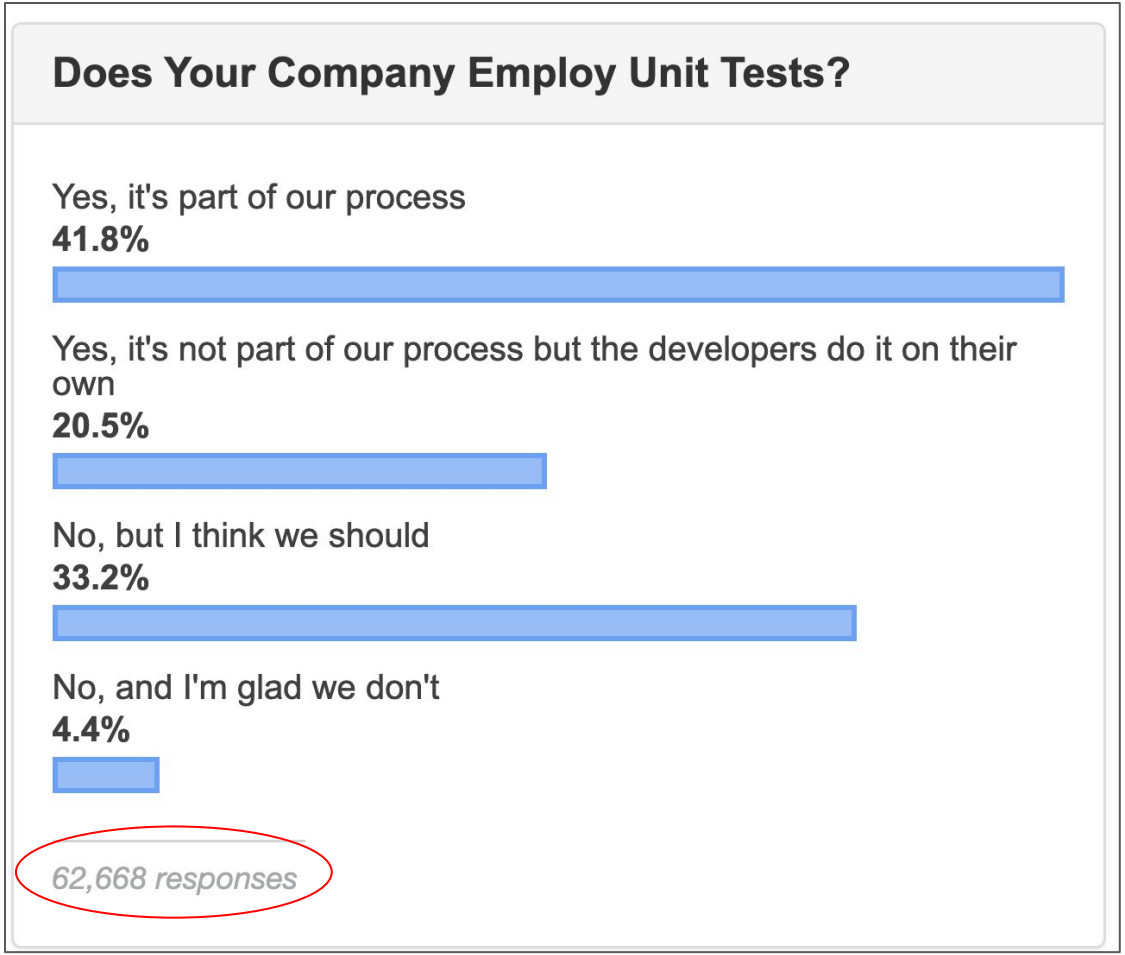
"No Facebook, engenheiros são responsáveis pelos testes de unidade de qualquer código novo que eles desenvolvam."



"Código sem testes é código ruim."

-- Michael Feathers

Stack Overflow Survey 2019



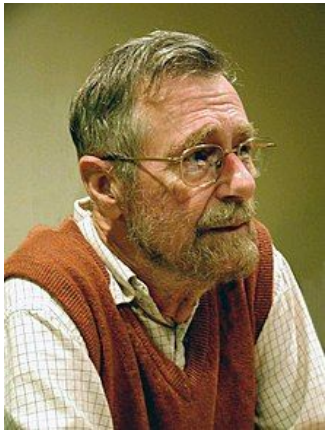
https://survey.stackoverflow.co/2019#development-practices

Relembrando Cap. 1 (Introdução)

Testes de Software

- Verificam se um programa apresenta um resultado esperado, ao ser executado com alguns casos de teste
- Podem ser:
  - Manuais
  - **Automatizados** (nosso foco)

Testes de software mostram a presença de bugs,
mas não a sua ausência.
-- Edsger W. Dijkstra



principal limitação

principal objetivo

Defeitos, Bugs, Falhas

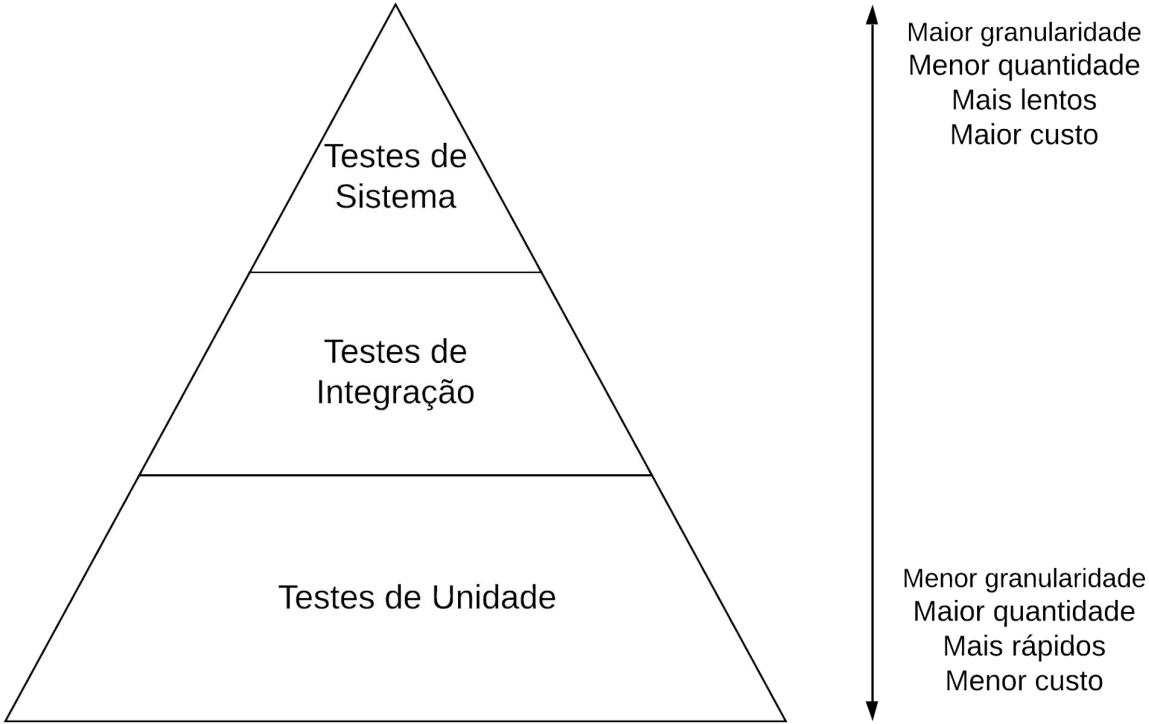
- Código com **defeito** (*defect*) ou **bug**
- Quando for executado ele vai causar uma **falha** (*failure*); ou seja, um resultado errado.

```
if (condition)
    area = pi * raio * raio * raio;
```

Testes com Métodos Ágeis

- Automatizados
- Escritos pelo próprio desenvolvedor do código sob testes

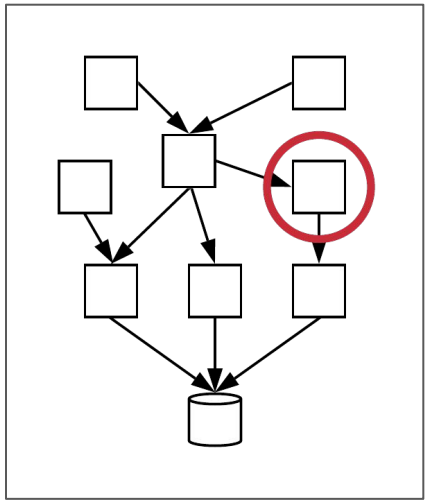
Pirâmide de Testes



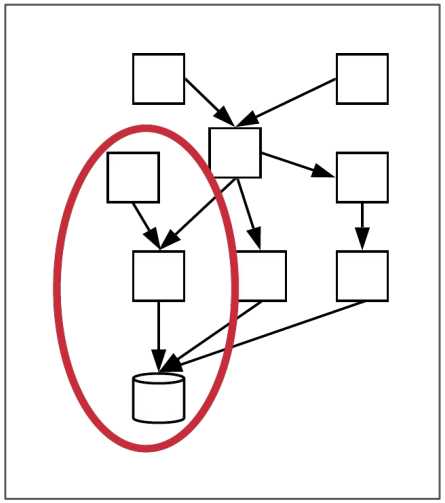
Maior granularidade
Menor quantidade
Mais lentos
Maior custo

Menor granularidade
Maior quantidade
Mais rápidos
Menor custo

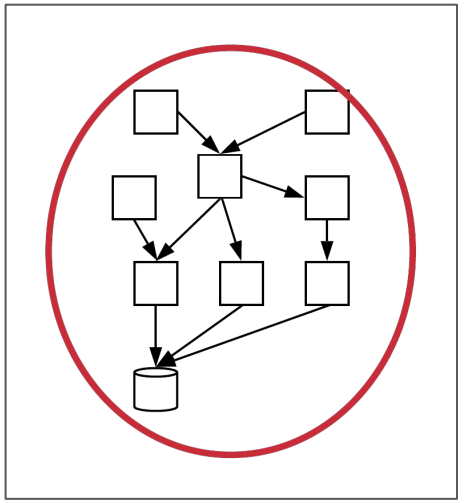
Tipos de Teste



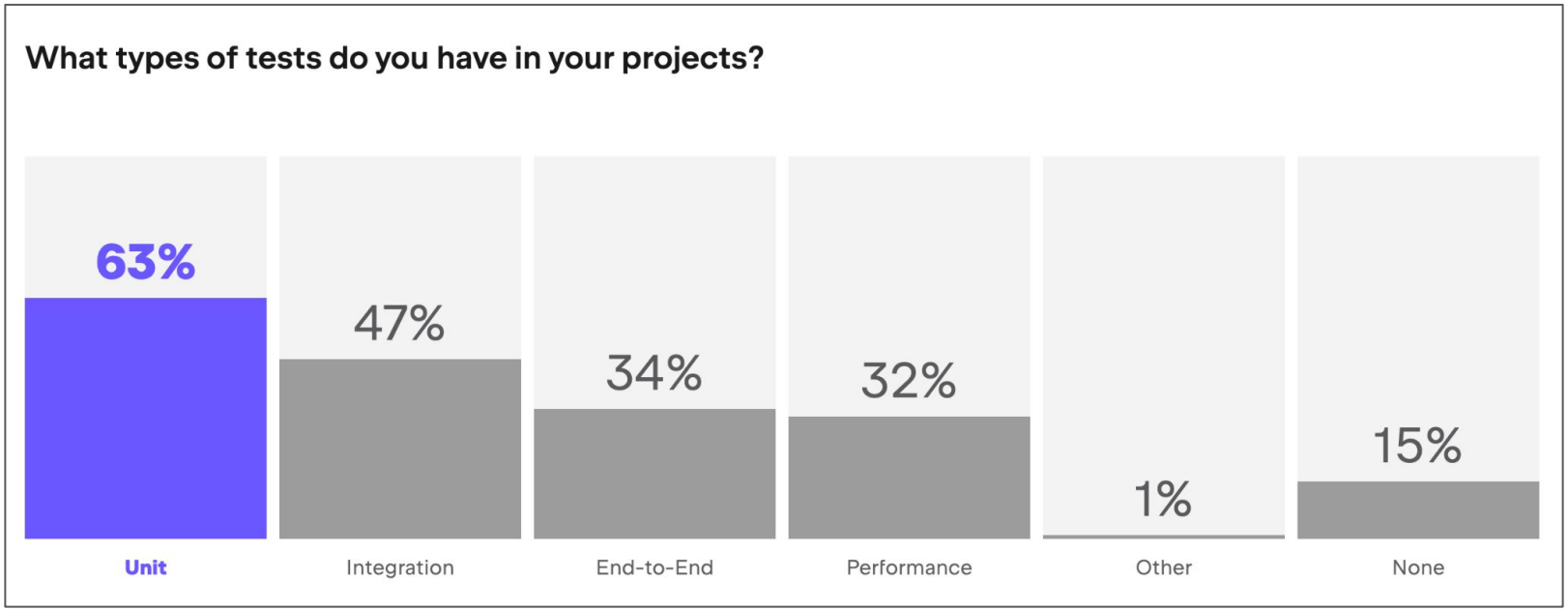
Unidade



Integração



Sistema (E2E ou End-to-End)



https://www.jetbrains.com/lp/devecosystem-2023/testing/

Testes de Unidade

Testes de Unidade

- São testes automatizados de pequenas unidades de código (normalmente, classes)

Primeiro Exemplo: teste de unidade para uma classe Stack

Classe Stack

```
public class Stack<T> {

  private ArrayList<T> elements =  new
ArrayList<T>();
  private int size = 0;

  public int size() {
    return size;
  }

  public boolean isEmpty() {
    return (size == 0);
  }

  public void push(T elem) {
    elements.add(elem);
    size++;
  }

  public T pop()  throws EmptyStackException {
    if (isEmpty())
      throw new EmptyStackException();
    T elem = elements.remove(size-1);
    size--;
    return elem;
  }
}
```

Classe Stack

```
public class Stack<T> {

  private ArrayList<T> elements =  new
ArrayList<T>();
  private int size = 0;

  public int size() {
    return size;
  }

  public boolean isEmpty() {
    return (size == 0);
  }

  public void push(T elem) {
    elements.add(elem);
    size++;
  }

  public T pop()  throws EmptyStackException {
    if (isEmpty())
      throw new EmptyStackException();
    T elem = elements.remove(size-1);
    size--;
    return elem;
  }
}
```

Classe StackTest

```
import org.junit.Test;
import static org.junit.Assert.assertTrue;

public class StackTest {

  @Test
  public void testEmptyStack() {
    Stack<Integer> stack =  new
Stack<Integer>();
    boolean empty = stack.isEmpty();
    assertTrue(empty);
  }

}
```

Anatomia de um Teste de Unidade

```
import org.junit.Test;
import static org.junit.Assert.assertTrue;

public class StackTest {

  @Test
  public void testEmptyStack() {
    Stack<Integer> stack = new Stack<Integer>();
    boolean empty = stack.isEmpty();
    assertTrue(empty);
  }

}
```

Métodos de teste (sem parâmetros, começam com test)

Fixture (contexto)

Chama **método sob teste**

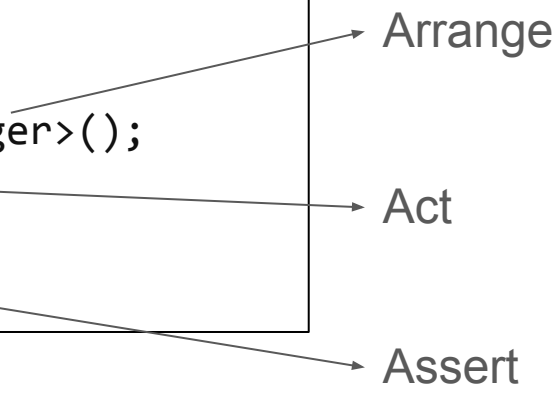
Comando **assert**: verifica se resultado é aquele esperado; se não for, lança uma exceção

Padrão AAA

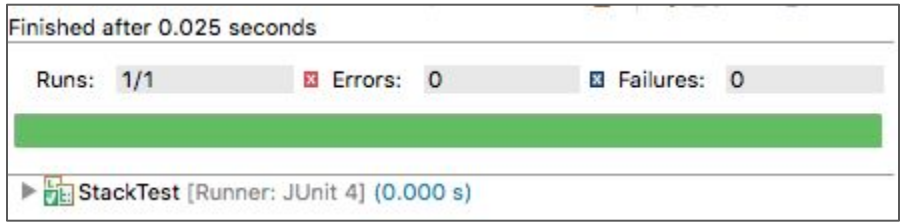
- Arrange: configura o ambiente de teste, inicializa variáveis, instancia objetos e cria mocks ou dados necessários.
- Act: executa a ação ou chama o método/função que está sendo testado.
- Assert: verifica se o resultado obtido corresponde ao comportamento ou valor esperado.

Padrão AAA

```
@Test
public void testEmptyStack() {
  Stack<Integer> stack = new Stack<Integer>();
  boolean empty = stack.isEmpty();
  assertTrue(empty);
}
```

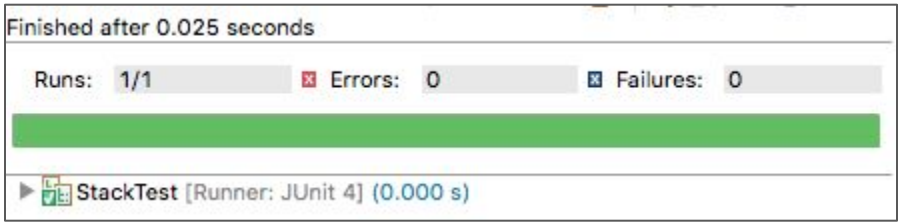


Framework de testes: xUnit

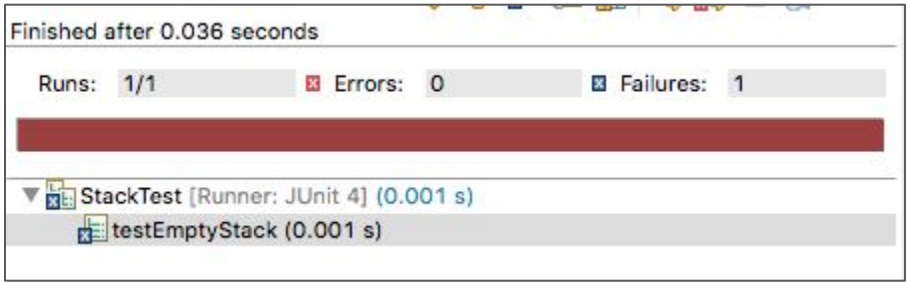


Testes passaram!

Framework de testes: xUnit



Testes passaram!



Algum teste falhou!

Frameworks para outras linguagens

- Python: unittest, pytest
- Ruby: RSpec
- JavaScript: Jest
- C: CUnit
- C++: CppUnit
- C#: NUnit

Todos muito semelhantes com JUnit

Mais métodos de teste

```
public class StackTest {

  Stack<Integer> stack;

  @Before
  public void init() {
    stack = new Stack<Integer>();
  }

  @Test
  public void testEmptyStack() {
    assertTrue(stack.isEmpty());
  }

  @Test
  public void testNotEmptyStack() {
    stack.push(10);
    assertFalse(stack.isEmpty());
  }
```

Executado antes de qualquer método @Test

```java
@Test
public void testSizeStack() {
  stack.push(10);
  stack.push(20);
  stack.push(30);
  int size = stack.size();
  assertEquals(3, size);
}

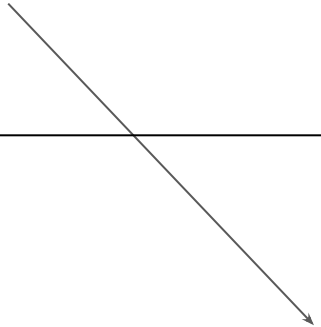
@Test
public void testPushPopStack() {
  stack.push(10);
  stack.push(20);
  stack.push(30);
  int result = stack.pop();
  result = stack.pop();
  assertEquals(20, result);
}
```

Parâmetros: valor esperado (3) e valor encontrado (size), nesta ordem.

Mensagem quando o assert falha:

Expected 3 but found [valor]

```
  @Test(expected = java.util.EmptyStackException.class)
  public void testEmptyStackException() {
    stack.push(10);
    int result = stack.pop();
    result = stack.pop();
  }
}
```

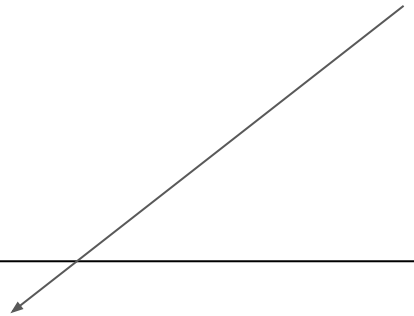


assert não seria útil aqui; pois ele não seria alcançado.

Novidades no JUnit 5

- `Before` renomeado para `BeforeEach`
- `assertThrows`: Nova asserção para testar exceções

```
@Test
public void testEmptyStackException() {
  assertThrows(java.util.EmptyStackException.class, () -> {
    stack.push(10);
    int result = stack.pop();
    result = stack.pop();
  });
}
```



Função lambda sem parâmetros; permite passar uma função como parâmetro de uma outra função (no caso, assertThrows)

Código de assertThrows (versão simplificada)

```
Throwable assertThrows(Class expectedException, Executable executable) {
  try {
    executable.execute();
  }
  catch (Throwable exception) {
    if (expectedException.isInstance(exception)) {
       return exception;
    }
    else {
      throw assertionFailure();
    }
  }
  throw assertionFailure();
}
```

Código dos Exemplos

https://github.com/mtov/ESM-ExemplosCodigo/tree/master/cap8

Mais alguns conceitos sobre testes

Alguns conceitos sobre testes

Benefícios

- Detectar bugs
  - Na classe C sendo implementada ou modificada
  - Em outra classe; mudança em C pode afetar C' (regressão)
- Documentação

Princípios FIRST (boas características de testes de unidade)

- Rápidos (**F**ast)
- **I**ndependentes (ordem não é importante)
- Determinísticos (**R**epeatable, não-flaky ou não-erráticos)
- Auto-verificáveis (**S**elf-checking): barra verde ou vermelha
- **T**imely (escritos o quanto antes)

Testes Flaky

- Não-determinísticos: algumas vezes passam, algumas vezes falham



Resultados de execuções de um um mesmo teste T em um programa que não sofreu modificação:

Duas perguntas

- Por que testes flaky são ruins?
- O que faz um teste ter um comportamento flaky?

Por que alguns testes tem comportamento flaky?

Fonte: An Empirical Analysis of Flaky Tests, FSE 2014.

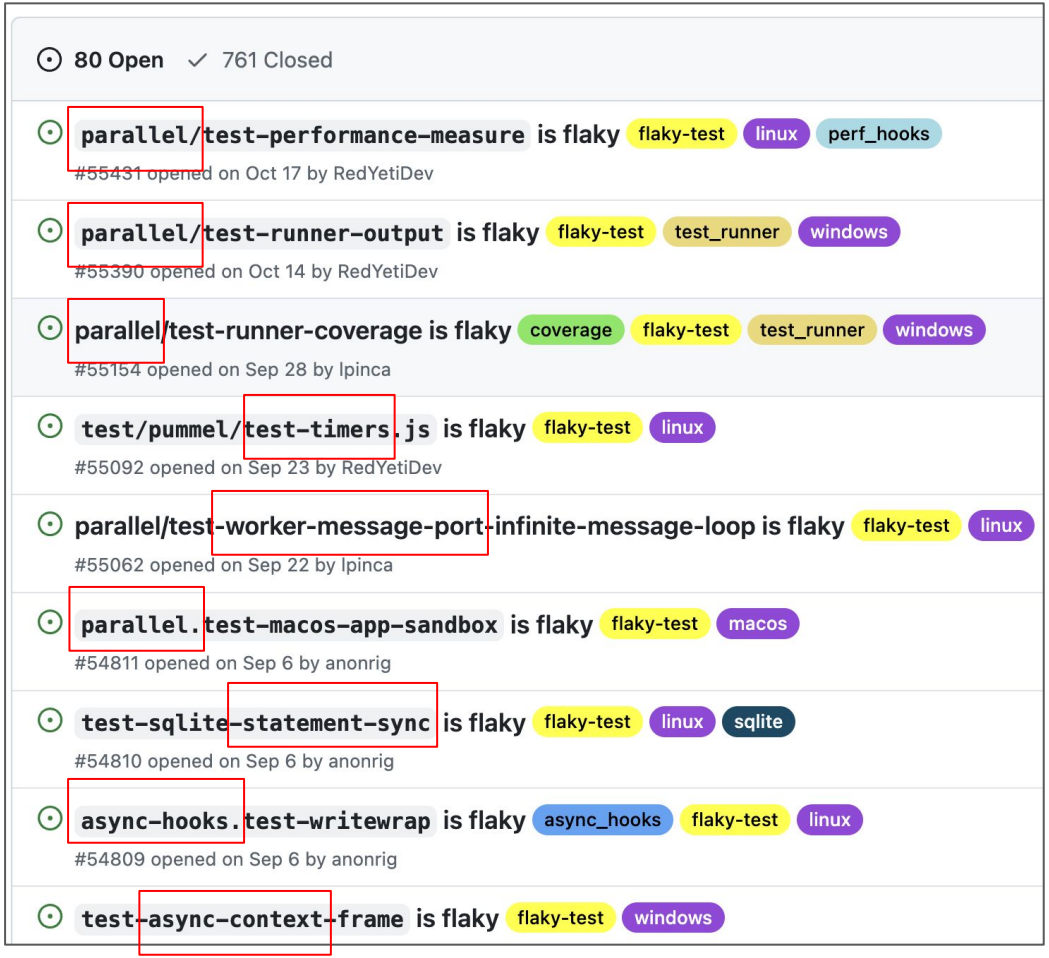
```
1 @Test
2 public void testRsReportsWrongServerName() throws Exception {
3   MiniHBaseCluster cluster = TEST_UTIL.getHBaseCluster();
4   MiniHBaseClusterRegionServer firstServer =
5     (MiniHBaseClusterRegionServer)cluster.getRegionServer(0);
6   HServerInfo hsi = firstServer.getServerInfo();
7   firstServer.setHServerInfo(...);
8
9   // Sleep while the region server pings back
10  Thread.sleep(2000);
11  assertTrue(firstServer.isOnline());
12  assertEquals(2,cluster.getLiveRegionServerThreads().size());
13  ... // similarly for secondServer
14 }
```

Concorrência

(65% dos casos)

E se "region server" demorar mais de 2s para dar o "ping"?

Exemplos de Testes Flaky - Node



https://github.com/nodejs/node/labels/flaky-test

Exercícios

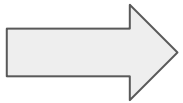
1. Se testes E2E testam “o mais importante” (isto é, o sistema inteiro), porque não é recomendado implementar apenas este tipo de teste?

Número de assert por testes

- Maioria das vezes: um único assert / teste

❌

```
@Test
public void testEmptyStack() {
  assertTrue(stack.isEmpty());
  stack.push(10);
  assertFalse(stack.isEmpty());
}
```



✅

```
@Test
public void testEmptyStack() {
  assertTrue(stack.isEmpty());
}

@Test
public void testNotEmptyStack() {
  stack.push(10);
  assertFalse(stack.isEmpty());
}
```

Mas existem exceções…

```
@Test
public void testRepeat() {
  String input = "20";
  assertEquals("", Strings.repeat(input,0));
  assertEquals("20", Strings.repeat(input,1));
  assertEquals("2020", Strings.repeat(input,2));
  assertEquals("202020", Strings.repeat(input,3));
  ...
}
```



Quantos testes eu tenho que escrever?

Cobertura de Testes

- Cobertura de testes = (número de comandos executados pelos testes) / (total de comandos do programa)

```java
public class Stack<T> {
  private ArrayList<T> elements = new ArrayList<T>();
  private int size = 0;

  public int size() {
    return size;
  }

  public boolean isEmpty(){
    return (size == 0);
  }

  public void push(T elem) {
    elements.add(elem);
    size++;
  }

  public T pop() throws EmptyStackException {
     if (isEmpty())
        throw new EmptyStackException();
     T elem = elements.get(size-1);
     size--;
     return elem;
  }
}
```

100% de cobertura

```java
public class Stack<T> {
  private ArrayList<T> elements = new ArrayList<T>();
  private int size = 0;

  public int size() {
    return size;
  }

  public boolean isEmpty(){
    return (size == 0);
  }

  public void push(T elem) {
    elements.add(elem);
    size++;
  }

  public T pop() throws EmptyStackException {
    if (isEmpty())
      throw new EmptyStackException();
    T elem = elements.get(size-1);
    size--;
    return elem;
  }
}
```

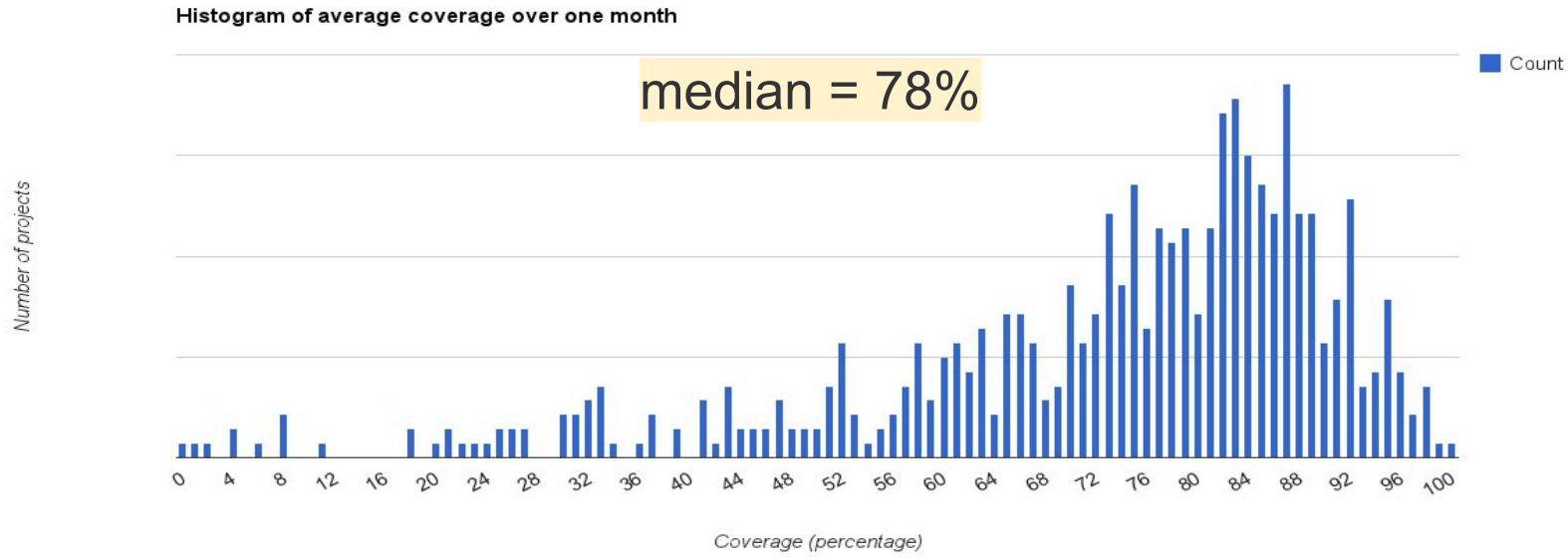
amarelo: apenas um dos branches do comando de decisão é avaliado

vermelho: comando não coberto pelos testes

Qual a cobertura de testes ideal?

- Varia de projeto para projeto, mas não precisa ser 100%
- Pelo menos 60%, segundo alguns autores

Exemplo: Google



https://docs.google.com/presentation/d1god5/fDDd1aP6PwhPodOnAZSPpD80lqYDrHhuhyD7Tvg/edit#slide=id.g3f5c82004_99_135

Cobertura de Branches

- Todo commando "if" possui dois branches: V e F
- Cobertura de branches = (número de branches executados pelos testes) / (total de branches do programa)

O próximo slide descreve um cenário que mostra as vantagens de cobertura de branches (em relação a cobertura de comandos)

- Suponha uma função que calcula descontos:

```
double calcularDesconto(preco) {
  if (preco > 1000)
     desconto = 0.1
  return desconto
}
```

- Porém, existe uma segunda regra que você esqueceu de implementar: se preco <= 1000, desconto = 0.05
- Se testar apenas com preco > 1000, a cobertura de comandos já será 100% e você pode ter a falsa impressão de que está tudo ok
- Já a cobertura de branches, será de 50%... Então, você pode ser incentivado a escrever um teste com preco < 1000, o qual vai te ajudar a encontrar o bug

Exercícios

1. Em uma universidade, os alunos recebem conceito A se tiverem nota maior ou igual a 90. A seguinte função foi usada, em um dos sistemas da universidade para implementar esse requisito:

```
boolean isConceitoA(int nota) {
  if (nota > 90)
    return true;
  else return false;
}
```

(a) Essa implementação possui um bug ? Se sim, quando esse bug resulta em falha?

(b) Suponha que a função seja testada com as notas 95 e 85. Qual a cobertura de comandos do teste? E a de branches?

2. Seja a afirmação:

> Se um programa possui 100% de cobertura de testes, em nível de comandos, ele está livre de bugs.

Ela é verdadeira ou falsa? Justifique.

3. Por que, normalmente, não é necessário e obrigatório atingir 100% de cobertura de comandos?



Professor Marco Tulio Valente

Slides disponibilizados pelo Prof. Marco Tulio Valente. Esta apresentação contém adaptações da apresentação original que está disponível na seguinte página: https://engsoftmoderna.info.

HORSTMANN, C.

Padrões e Projeto Orientados a Objetos, 2 Edição, Editora Bookman: Porto Alegre, 2007

BEZERRA, E.

Princípios de Análise e Projeto de Sistemas com UML, 2a ed., Editora Campus/Elsevier: Rio de Janeiro, 2007

SOMMERVILLE, I.

Engenharia de Software. 8a ed., Ed. Pearson: São Paulo, 2008.

PAULA, Filho W. P.

Engenharia de Software: Fundamentos, Métodos e Padrões, 3a ed. Ed. LTC: Rio de Janeiro, 2009.

PRESSMAN, R.

Software Engineering: A Practitioner's Approach. 5a ed. McGraw Hill : São Paulo, 2000

